

1.1 Creating and Populating Tablets with Constraints

1. Constraints Check

- (1) A NOT NULL constraint is always declared as a column constraint.
- (2) Referenced column required to be primary key for portability.
- (3) We omit the check if any value in the set to be checked is NULL.
- 注意考试的时候除了检查的还要**检查数据类型!!!**
2. **Transaction and Deferred**
Transaction 事务: BEGIN, END 或 COMMIT, ABORT 和 ROLLBACK, BEGIN...COMMIT 控制事务范围。
BEGIN;
INSERT INTO child (parent_id, description) VALUES (1, 'child 1');
INSERT INTO parent (id, name) VALUES (1, 'parent 1');
COMMIT;
Deferred 延迟检查: 在大多数系统中 integrity constraints 是同时检查的。理想的做法是将所有完整性的设置为延迟检查 (即在事务结束时再进行检查) deferrable
3. **两类删除**
DELETE FROM customers; --DELETE deletes the content of the table but NOT its definition.
DROP TABLE customers; -- DROP deletes the content and the definition of the table.
4. **更新**
Cascade:
ON DELETE/UPDATE OP
(1) CASCADE: 表删改我也删改
(2) NO ACTION: 删改不能变
(3) SET DEFAULT: 表变化后子表变为默认值
(4) SET NULL: 表变化后子表变为 null
表的修改:
UPDATE 表名
SET 列 1 = 新值 1, 列 2 = 新值 2, ...
WHERE 条件;

1.2 SIMPLE QUERIES

1. SELECT FROM-WHERE

2. ORDER BY
ASC(default) and DESC
We can order the result according to a column that is **not** printed. But this may interfere with other constructs. Only used this when you are sure that it will work.
3. DISTINCT
Incompatible

```
SELECT DISTINCT name, version
FROM games
ORDER BY price ASC;
```

```
Error
price does not appear in DISTINCT.
```

- 这里的问题在于多个重复的 (name, ver) 可能对应不同的 price, order by 无法确定根据哪个 price 进行排序
4. **Condition**
SELECT first_name, last_name
FROM customers
WHERE country IN ('Singapore', 'Indonesia')
AND (dob BETWEEN '2000-01-01' AND '2000-12-01' OR since >= '2016-12-01')
AND last_name LIKE 'B%';
(1) **Like 与字符串匹配**
% 表示任意长度的字符 (包括 0 个字符)。
_ 表示单个字符。

Week6 Functional Dependency

1. FD 依赖:

- 最小实例: 1. valid: empty; 2. valid: 2 rows
Trivial: X=Y is trivial if and only if Y is subset of X
非平凡但完全非平凡的 FD, 可拆分为 1 个平凡 FD 和 1 个完全非平凡 FD (如 (A)→B(C) 可拆为 (A)→(C) (平凡) 和 (A,C)→(B) (完全非平凡))

2. KEY 键:

- (1) Superkey: 能够唯一确定一行记录
- (2) Candidate key: 再少一个 attribute 就不能唯一确定一行记录的 superkey (最小的 superkey)
- (3) Prime attribute: 出现在 candidate key 中的 attribute
任意关系表 R 至少有 1 个超键 (如 R 本身, 因 R→R 是平凡 FD)

考点: 求 candidate key

- (1) 记关系表的属性全集 U 和所有函数依赖集 F。
- (2) 将 F 化为最小覆盖 FD
- (3) 找出 **不会出现在任一 FD 右侧 (RHS) 的属性集合 M**。这些属性必须出现在每个候选键中 (因为它们不能被别的属性推出)。
- (4) 从 M 出发, 尝试加入其它属性组合, 计算该集合在 FD 下的属性闭包
- (5) 若某组合的闭包等于 U, 则该组合是属性。继续检查最小性: 若任何真子集仍是属性, 则丢弃 (不是候选键)
- (6) 将所有满足的最小超键列为候选键。

3. Closure 闭包

- 函数依赖闭包 (Σ⁺): Σ 逻辑蕴含的所有 FD 的集合 (含 Σ 本身及可推导的 FD) 属性闭包 (Σ⁺): 由属性集 S 能确定的所有属性的集合 (S-Σ⁺)
Armstrong 公理:
自反性: 若 X→Y, 则 X→Y (A)→(A), (C,D)→(D)
增广性: 若 X→Y, 则 XZ→YZ (Z∈S) 由 A→B, 得 AUC→BUC, AUD→BUD
传递性: 若 X→Y 且 Y→Z, 则 X→Z 由 C→A 且 A→B, 得 C→B
考点: 属性闭包算法

考点: 属性闭包算法 (Algorithm #1.2)

- 输入: 属性集 S, FD 集 Σ
输出: S 的属性闭包 S⁺
(1) 初始化: α=Z, 未使用的 FD, Γ=Σ (当前闭包)。
(2) 迭代: 若存在 FD X→Y 且 X⊆Γ, 则 α=α∪(X-Y), Γ=Γ∪Y; 重复此步骤直至 α 中无满足条件的 FD。
(3) 返回 Γ (即 S⁺)

4. FD 的等价与覆盖

- 等价:** 若 FD 集 Σ₁ 的闭包 Σ₁⁺=Σ₂⁺, 则 Σ₁ 与 Σ₂ **等价**, 称 Σ₁ 是 Σ₂ 的覆盖
Σ₁ 与 Σ₂ 等价, 当且仅当 Σ₁ 中的每个 FD 都蕴含于 Σ₂, 且 Σ₂ 中的每个 FD 都蕴含于 Σ₁

最小覆盖 Minimal covers

- 右部最小: 每个 FD 的右部仅含 1 个属性 (如 (A→B,C) 拆为 A→B, A→C)。
左部最小: 每个 FD 的左部不可移除任何属性 (移除后, 剩余左部无法蕴含原 FD)。
集合最小: 不可从 Σ_c 中移除任何 FD (移除后, 剩余集合与原 Σ 不等价)。
每个 FD 都至少含 1 个最小覆盖, 且可能不唯一。 **同一个 FD 的两个最小覆盖可能没有任何一个相同的依赖。**

考点: 最小覆盖算法 (Algorithm #2.2)

- (1) 简化右部: 将所有 FD 的右部分解为单一属性, 得到 Σ₁。
- (2) 简化左部: 对 Σ₁ 中每个 FD X→A, 尝试移除 X 中的每个属性 B, 若 (X-B) 包含 A, 则用 (X-B) 替换 X→A, 得到 Σ₂。
- (3) 简化集合: 对 Σ₂ 中每个 FD X→Y, 若移除 X→Y 后的集合仍能蕴含 X→Y, 则移除该 FD, 得到 Σ₃ (最小覆盖)。

规范覆盖 Canonical covers

- 在最小覆盖 Σ_c 的基础上, 合并所有左部相同的 FD, 得到规范覆盖 Σ_{nc}。

考点: 规范覆盖算法 (Algorithm #2.3)

- (1) 找出最小覆盖的 3 个步骤, 得到 Σ_c。
- (2) 合并 Σ_c 中左部相同的 FD (如 (A→B, A→C) 合并为 (A→B,C)), 得到 Σ_{nc}。

Week9 BCNF

1. Boyce-Codd 范式 (BCNF)

- 定义:** 非键属性必须提供与键 (或所有键) 相关的事实, 且必须与键的全部内容相关。除此之外, 还必须包含任何其他无关信息。(非键属性仅描述键)

考点: BCNF 的必须:

- 当且仅当对于函数依赖集 Σ 的闭包 Σ⁺ 中的每一个函数依赖 X→(A), 满足以下两个条件之一时, 具有该函数依赖集 Σ 的关系 R 才属于 Boyce-Codd 范式 (BCNF):
1. X→(A) is trivial;
2. X is a superkey.

- 注: X 不能是 candidate key 的子集: 通常, 这表明属性集 X 和属性 Y 属于某个实体, 而该实体与其余属性无关。

2. 表的分解

- 2.1 分解 decomposition:** 将 R 拆分为 n 个表 R₁~R_n, 满足 R=R₁∪...∪R_n
分解的性质: 无损连接、依赖保持

(2) Between And 包含相等的情况

```
SELECT name || ' ' || version,
CASE
WHEN price < 0.09 >= 0.3 THEN ROUND(price < 1.09, 2)
ELSE price
END AS price
FROM games;
```

(3) Operators

- Select 语句和 where 子句均可以包含计算;
|| 是 字符串拼接运算符, 用于将 name 数据和 version 数据中间加个 ' ' 连接起来;
CASE-WHEN-THEN-ELSE: We perform the first condition that is satisfied with checking done from top to bottom.
5. **Logic and NULL**
NULL means unknown
Sth. Op NULL = NULL even if Sth. Is NULL
注意 NULL > NULL evaluates to NULL
(1) IS NULL, IS NOT NULL checks for null values
(2) COALESCE(x1, x2, ...) 返回第一个不是 NULL 的值
意思是如果 col2 是 null 就返回 col1, 否则就返回原本的值
(3) **聚合函数与 NULL:** COUNT(*) 会计入为 NULL 的行
COUNT(ATT), AVG(ATT), MAX(ATT), MIN(ATT) 会清除 NULL 的行

6. **多表连接**
(1) **Inner join:** 如果某一行在另一张表里找不到匹配, 就不会出现在结果里 (适用于一张表没有就干脆不出现)
(2) **Outer join:** 返回左 (右) 表所有行, 如果右 (左) 表中没有匹配则结果为 NULL (适用于不管有没有都得出现)

- (3) **Natural join:** 自动按同名的列进行连接 (笛卡尔积), 不需要 ON 条件 (适用于有同名列)
(4) **Cross join:** 笛卡尔积, 结果行数 = 表 A 行数 × 表 B 行数
SELECT a.col, b.col
FROM tableA a
CROSS JOIN tableB b;

L3 AGGREGATE QUERY

1. **Aggregation**
COUNT(*), COUNT(ATT), SUM(ATT), MAX(ATT), MIN(ATT), AVG(ATT), STDDEV(ATT)
COUNT(DISTINCT col)
TRUNK(AVG(col), n) n 表示保留的位数

- 注意:
表中除了 COUNT 以外的函数均返回 NULL, COUNT 的两种情况均返回 0
在非空全 NULL 表中, 除了 COUNT 以外的函数均返回 NULL, COUNT(ATT) 返回 0, COUNT(*) 返回总行数

2. **Group By**
It is recommended (and required per SQL standard) to include attributes projected in the SELECT clause to the GROUP BY clause.
只有姓名都是 unique 字段才能执行
只有姓名没有 ID 是错的, 不知道姓名的选哪个 ID

```
SELECT c.customerid, c.first_name, c.last_name, SUM(g.price)
FROM customers c, downloads d, games g
WHERE c.customerid = d.customerid
AND d.name = g.name AND d.version = g.version
GROUP BY c.customerid;
```

3. **Having**
Having clause can only use aggregate functions, columns listed in the GROUP BY clause, and subqueries.

4. **Order By**
可以用 Order By 指定用来分组的字段的排列方式

5. **执行顺序**
FROM → WHERE → GROUP BY → HAVING → SELECT →

ORDER BY → DISTINCT

这个涉及到 AS 别名 的适用范围

6. 代码顺序

- SELECT → FROM → JOIN → WHERE → GROUP BY → HAVING
→ ORDER BY
7. 集合操作
并集 (UNION)、交集 (INTERSECT) 和非对称差集 (EXCEPT) 在默认情况下会去除重复行, 除非使用关键字 ALL (UNION ALL)
两个查询必须在结构上兼容 (union-compatible), 才能用于 UNION、INTERSECT 或 EXCEPT

L4 ER-Diagram

1. Cardinality

2. **Schema translation**
The candidate keys are mapped to **primary keys** and **UNIQUE NOT NULL**
集合本质上就是一个关系集, 因此适用于关系集的规则。但它也在某种意义上也可以作为一个实体集使用, 因为它的键可以被其他关系集引用。

有一些需要注意的点:

- (1) 如果一对多关系, 大部分时候, 关系表直接并入多的一方实体表, 少数情况下也可以作为一个实体集使用, 可以单独建立关系表, 但是转化成 schema 时主键只有多的一方为主键, 不要与联合主键

```
CREATE TABLE work_for (
start DATE NOT NULL,
end DATE NOT NULL,
number CHAR(3) PRIMARY KEY,
cname VARCHAR(12) NOT NULL,
-- NOT NULL to ensure existence
FOREIGN KEY (number)
REFERENCES employee(number),
FOREIGN KEY (cname)
REFERENCES company(name)
);
```

```
CREATE TABLE employee_work_for (
start DATE NOT NULL,
end DATE NOT NULL,
enumber CHAR(3) PRIMARY KEY,
fname VARCHAR(12) NOT NULL,
cname VARCHAR(12) NOT NULL,
FOREIGN KEY (enumber)
REFERENCES employee(number),
FOREIGN KEY (cname)
REFERENCES company(name)
);
```

- (2) 如果是强制一对一关系, 需要保证最小的参与量, 也就是至少有 1 参与, 则外键连接的另一实体必须非空

(3) 弱实体使用联合主键

L5 NESTED QUERY

1. Copy 复制表

```
CREATE TABLE singapore_customer AS
SELECT *
FROM customers c
WHERE c.country = 'Singapore';
CREATE TEMPORARY TABLE singapore_customer AS
SELECT *
FROM customers c
WHERE c.country = 'Singapore';
```

2. **Query 查询表**
2.1 **View**
2.2 **CTE**
Common table expression is a copy of a subquery in a temp table that only exists for the query.

```
WITH singapore_customer AS
(SELECT *
FROM customers c
WHERE c.country = 'Singapore')
SELECT cs.last_name, d.name
FROM singapore_customer cs, downloads d
WHERE cs.customerid = d.customerid;
```

```
CREATE VIEW singapore_customer AS
SELECT *
FROM customers c
WHERE c.country = 'Singapore';
```

2.3 From

```
SELECT cs.last_name, d.name
FROM ( SELECT *
FROM customers c
WHERE c.country = 'Singapore' ) AS cs, downloads d
WHERE cs.customerid = d.customerid;
```

We can use subquery in the SELECT clause, but it must return only one column and one row

(1) where ... in (nested query) 处理 null 时如果没有匹配的且值有 null 值返回 unknown
(2) where ... = any() 处理 null 时只看不是 null 的值后比较, 如果有 null 那就返回 unknown
(3) 找最大值: where ... >= any()
(4) 找最小值: where ... <= any()
(5) 哪里无法用聚合函数 max 和 min, 可以转化成:

(6) exist 的妙用 (p->q 的变形)

```
SELECT c.first_name, c.last_name
FROM customers c
WHERE NOT EXISTS (
SELECT *
FROM games g
WHERE g.name = 'Aerified'
AND NOT EXISTS (
SELECT *
FROM downloads d
WHERE d.customerid = c.customerid
AND d.name = g.name
AND d.version = g.version ));
```

2.6 **Correlation**
(1) **注意 scope:** 在 where 内层子查询里出现的表别名不能直接在外层查询的 select 里使用, 因为作用域只在内层
(2) select 里的子查询必须是标量子查询, 对一个 select 的其他属性, 只有唯一的一个子查询结果与之对应
(3) <> 的妙用: where ... not in; where ... <> all(); where not exists()
(4) 有聚合函数要使用 having 的嵌套

其他需要注意的东西:

1. 看清楚题目是符合题意还是不符合题意的
2. 对 T-name、id 之类的属性, 看清楚中指主体到底是哪一个
3. 有外键约束的表要注意, 考察是否能实现某些数据的插入时要考虑插入顺序
4. 明显具有 A-B 结构的查询语句, 要考虑 A 到底是否存在 (是否已有一个全集)
5. 画 ER 图的时候要考虑清楚参与量, 可以先理清主体间的关系, 对于弱实体, 要注意虚线, 要注意虚线
6. Schema 转 ER 图有些表可能是并没有被实现出来的

Week6 Relation Algebra

Operation	Symbol	Operation	Symbol	Operation	Symbol	Operation	Symbol
Selection	σ	Projection	π	Renaming	ρ	Cross Product	$\times$
Conjunction	$\wedge$	Disjunction	$\vee$	Negation	$\neg$	Join	$\bowtie$
Union	$\cup$	Intersection	$\cap$	Difference	$-$	Outer joins	$\ltimes, \ltimes^*, \ltimes^*$

1. **单元选择符**
• $\sigma[c]$ selects all rows from the relation R that satisfies the selection condition c
• $\pi[i]$ keeps only the columns specified in the ordered list i and in the same order
• $\rho(R1, R2)$ can be used to rename the relation

2. 集合

Find the different pizza sold by Bella Italia but NOT Desert Diner.
Q1 := $\pi[\text{pizza}][\sigma[\text{name} = 'Bella Italia'](\text{sell})]$
Q2 := $\pi[\text{pizza}][\sigma[\text{name} = 'Desert Diner'](\text{sell})]$
Q1 - Q2
当查询的结果是一个由一个全集减去一部分时可以考虑 except

3. 连接

- $R1 \times R2$ combine each row of R1 with each row of R2 and keep the n columns of R1 and the m columns of R2

```
 $\pi[cname, r.name]([
\sigma[c.area = r.area](
p(customer, c) \times p(\text{restaurant}, r)
)])$ 
```

1. $R1 \bowtie R2$ is simply defined as $\sigma[c](R1 \times R2)$. In other words, we include only tuples that satisfies the condition c after the cross product.

SELECT DISTINCT a1, a2, a3, ...
FROM r1 JOIN r2 ON c1
JOIN r3 ON c2
[a1, a2, a3, ...]
r1 $\bowtie$ c1 r2 $\bowtie$ c2 r3

- **Natural join:** If we exclude the condition c, the operator becomes the natural join operator (i.e., $\bowtie$). For example, $R1 \bowtie R2$.

- **Outer join:** We also have left $\bowtie$ (left), right $\bowtie$ (right), and full $\bowtie$ (full) outer join variants that depends on the condition c.

4. Universal Quantification

Print the emails and the names of the different students who borrowed all the books authored by Adam Smith

```
SELECT s3.email, s3.name
FROM student s3
EXCEPT
SELECT tmp.email, tmp.name
FROM (
SELECT s1.email, s1.name, b1.ISBN13
FROM student s1, book b1
WHERE b1.authors = 'Adam Smith'
EXCEPT
SELECT s2.email, s2.name, b2.ISBN13
FROM student s2, book b2, loan l2
WHERE b2.authors = 'Adam Smith'
AND s2.email = l2.borrower
AND b2.ISBN13 = l2.book
AND l.borrower = s3.email));
```

Q1 := $\pi[b1.ISBN13][\sigma[b1.authors = 'Adam Smith'](\rho(\text{book}, b1))]$
⇒ The ISBN13 of all books authored by Adam Smith.

Q2 := $\pi[s1.email, s1.name](\rho(\text{student}, s1)) \times Q1$
⇒ Relational algebra query for Step a. Note: The columns are [email, name, ISBN13].

Q3 := $\pi[s2.email, s2.name, b2.ISBN13](
\rho(\text{loan}, l2) \bowtie [s2.book = b2.ISBN13 \wedge b2.authors = 'Adam Smith'] \rho(\text{book}, b2))$
⇒ [l2.borrower = s2.email] $\rho(\text{student}, s2)$

⇒ Students who have borrowed books by Adam Smith.

Q4 := $\pi[s2.email, s2.name](Q2 - Q3)$
⇒ Relational algebra query for Step b followed by projection. This is also the final answer for Step 1. Note: Q2 and Q3 are union-compatible. Now, the columns are [email, name].

Q5 := $\pi[s3.email, s3.name](\rho(\text{student}, s3)) - Q4$
⇒ Note: The projection on s3 is to ensure union-compatibility with Q4.

FD: $X \rightarrow Y$, $X, Y \subseteq R$ such that $X \text{ agree} \Rightarrow Y \text{ agree}$. R satisfies $X \rightarrow Y$ is valid with Σ if all FD satisfied, otherwise violated. Σ holds on R . $\Sigma \models X$.

Non-trivial: $Y \neq X$. **Completely non-trivial:** $Y \neq \emptyset$ and $Y \cap X \neq \emptyset$. **Theorem 1:** A non-trivial FD can be split into a trivial + a completely non-trivial FD. **Superkey:** $S \rightarrow R$ (superset of key).

Theorem 2: R has at least 1 superkey. **Candidate key:** minimal superkey. $S \rightarrow R$ and any $T \subset S$ not superkey. **Prime attribute:** $\bigcup K_i$. Appears in some CK. **Closure:** Σ^+ logically entailed by Σ .

S^+ is set of all attributes FD on S . $S \rightarrow S^+$ and for all $Y \subseteq S^+$, $S \rightarrow Y$. **Armstrong axioms:** (i) Trivial FD are valid. [Reflexivity] (ii) (on add attributes on LHS + RHS. [Augmentation])

(iii) can short-circuit. [Transitivity] AA are sound + complete. **Theorem 3+4** **Equivalent:** $\Sigma_1 \equiv \Sigma_2 \Leftrightarrow \Sigma_1^+ \equiv \Sigma_2^+$. **Cover:** Same closure. **Theorem 5:** Equivalent $\Leftrightarrow \emptyset \in \Sigma$, logical consequence of Σ_2 , vice versa. **Minimal:** RHS minimal [Every FD $X \rightarrow \{A\}$]. LHS minimal $[X \rightarrow \{A\} \nrightarrow t. \text{ no } Y \subseteq \{A\}, Y \subset X]$. Set minimal [nothing derivable]. **Minimal cover:** Both minimal and cover. [RHS 1 attr, Can't remove LHS, Can't remove Σ] **Canonical cover:** Minimal cover + merge all LHS. **Theorem 6:** $\exists$ minimal cover (basis), not unique. For FD $X \rightarrow \{A\}$, $B \in X$ can be removed if $X - \{B\} \rightarrow \{A\}$ entailed by Σ . (Replace $X \rightarrow \{A\}$ by $X - \{B\} \rightarrow \{A\}$). $[A \in (X - B)^+]$ $X - Y$ can be removed from Σ if $X \rightarrow Y$ entailed by $\Sigma' - \{X \rightarrow Y\}$. (Replace Σ by $\Sigma' - \{X \rightarrow Y\}$).

Algorithm 1: Closure:
input S, Σ
output S^+
begin
 $\Omega := S$; [unused]
 $\Gamma := S$; [closure]
 while $X \rightarrow Y \in \Sigma$ and
 $X \subseteq \Gamma$ do
 $\Omega := \Omega \cup Y$;
 $\Gamma := \Gamma \cup Y$;
 return Γ

Algorithm 2: Minimal Cover:
input Σ
output Σ_c
1. Simplify RHS of FD in Σ produce Σ_A
2. Simplify LHS of FD in Σ_A produce Σ_B
3. Simplify set Σ_B to produce Σ_c
return Σ_c

Algorithm 3: Canonical Cover:
input Σ
output Σ_D
1. Run Minimal Cover on Σ to produce Σ_c
2. Regroup all FD with same LHS in Σ_c to produce Σ_D
return Σ_D

Algorithm: Projection:
1. List all subsets of attrs
2. Compute the closures
3. Intersect RHS with R'
4. Subtract LHS from RHS
5. Construct FDs
(e.g., $\{B\}^+ = \{D, E\}$ gives $\{B\} \rightarrow \{D, E\}$)

Algorithm 4: BCNF Decomposition:
Take $X \rightarrow Y \in \Sigma$ that violates BCNF. Decompose R into $R_1 := X^+$, $R_2 := (R - X^+) \cup X$.
Check R_1, R_2 with $\Sigma|_{R_1}, \Sigma|_{R_2}$ in BCNF. If not, recursion.
Theorem 7: It is lossless-join.
Theorem 8: It will terminate.

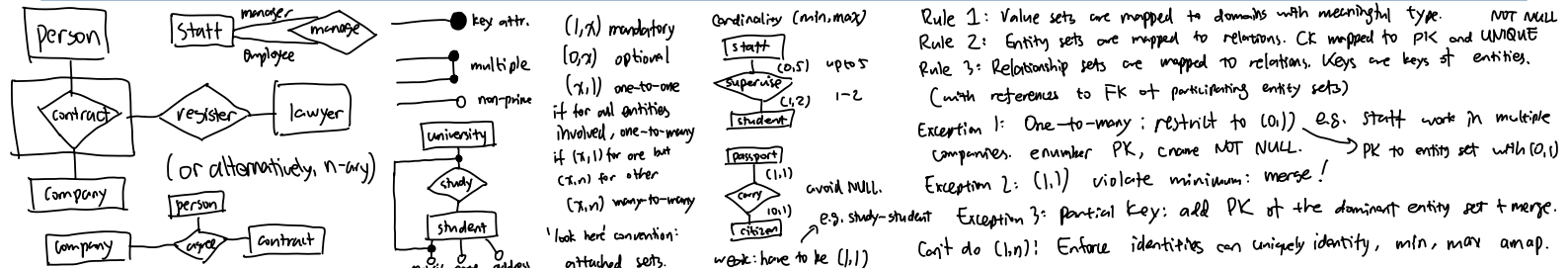
Algorithm 5: 3NF Synthesis:
For each $X \rightarrow Y \in \Sigma$ minimal cover, create $R_i := XY$. (Unless exists or subsumed). If key not contained, add a CK.
 $\neq$ a.k.a. Bernstein. $\neq$ BCNF?
 $\neq$ canonical also OK.
Theorem 9: It is lossless-join and dependency-preserving.

BCNF: (R, Σ) s.t. $\forall X \rightarrow \{A\} \in \Sigma^+$: trivial or X superkey. (Before decomposition, Σ sufficient). (Otherwise, X and A are part of an entity set unrelated to the rest).

Binary decomposition: $R = R_1 \cup R_2$. **(General) decomposition:** $R = R_1 \cup \dots \cup R_n$. (Do not lose any attr!) **Lossless-join:** FULL OUTER JOIN = initial. (Non-additive) Otherwise **lossy** (adds rows)

Lemma 1: $(R_1 \cap R_2) \rightarrow R_1$ or $(R_1 \cap R_2) \rightarrow R_2 \Rightarrow$ lossless-join. **Lemma 2:** (general) $\exists$ sequence of binary join. $[R_1 \cap R_2 \text{ PK} \Rightarrow \text{can be FK}]$. Steps: (i) Find $R_0 = R_1 \cap R_2$ (ii) Compute R_1^+ . (iv) $R_1 \subseteq R_0^+$ or $R_2 \subseteq R_0^+$? **Projected FDs:** Σ' on R' from (R, Σ) s.t. $R^+ \subseteq R'$ and $\forall X \rightarrow Y \in \Sigma^+$, $X \subseteq R'$ and $Y \subseteq R'$. Denoted as $\Sigma|_{R'}$.

i.e. $X \rightarrow Y$ entailed by R and X, Y are attr from R . [Note: check all FD in Σ^+ .] **Dependency-preserving:** $\Sigma^+ = (\Sigma|_{R_1} \cup \dots \cup \Sigma|_{R_n})^+ \Leftrightarrow$ d.p. Check $\emptyset \in \Sigma$ entailed by $(\Sigma|_{R_1} \cup \dots \cup \Sigma|_{R_n})$. (One side satisfies as $\Sigma \models$ all $\emptyset$) **3NF:** (R, Σ) s.t. $\forall X \rightarrow \{A\} \in \Sigma^+$: trivial or X superkey, or A prime attribute. **Abstract 1:** $R = \{A, B, C, D, E, F\}$ with Σ . R not in 3NF, max number of keys? $\binom{5}{2} = 10$. For $|R| = n$: $\binom{n-1}{2-1}$. **Abstract 2:** key equal $\Rightarrow$ FD equivalent. **Abstract 3:** minimal cover might have no intersection. Δ



Question

Find all the different pairs of customer name (*cname*) and pizza that the customer likes such that the price of the pizza is more than 15.

SQL

```
SELECT DISTINCT l.cname, s.pizza
FROM likes l, pizza p, sells s
WHERE l.pizza = p.pizza
AND s.pizza = p.pizza
AND s.price > 15;
```

Relational Algebra

```
π(l.cname, s.pizza)(
σ(l.pizza = p.pizza
  ∧ s.pizza = p.pizza
  ∧ s.price > 15)(
  l ⋈ p ⋈ s
))
```

Find the different customers that likes all the pizza. Include pizza that is not sold by any restaurant.

SQL

```
SELECT c1.cname
FROM customer c1
EXCEPT
SELECT t.cname
FROM (
  SELECT c.cname, p.pizza
  FROM customer c, pizza p
  SELECT * FROM likes l
) AS t;
```

Relational Algebra

```
Q1 := π(c.cname, p.pizza)(p(customer, c)
  × p(pizza, p))
Q2 := Q1 - likes
Q3 := π(c.cname)(Q2)
```

$π(c1.cname)(p(customer, c1)) - Q3$

Find the different pair of customer name (*cname*) and pizza the customer like such that the customer is in London and the pizza cost is less than 20.

SQL

```
SELECT DISTINCT cname, pizza
FROM customer
NATURAL JOIN likes
NATURAL JOIN pizza
NATURAL JOIN sells
WHERE area = 'London'
AND price < 20;
```

Relational Algebra

```
π(cname, pizza)(
σ(area = 'London' ∧ price < 20)(
  customer ⋈ likes ⋈ pizza ⋈ sells
))
Q1 := p(customer, c) × p(likes, l) × p(pizza, p) × p(sells, s)
Q2 := σ(c.cname = l.cname ∧ l(
Q1))
```

(b) Let us check the integrity of the data. Find the emails of the students who borrowed or lent a copy of a book before they joined the university. There should not be any.

Comments: One possible relational algebra expression.

```
π(s.email)(
σ(s.email = l.borrower ∨ s.email = l.owner) ∧ (l.borrowed < s.year)(
  p(student, s) × p(loan, l)
))
```

Code: Alternative #1

```
SELECT s.email
FROM student s, loan l
WHERE (s.email = l.borrower OR s.email = l.owner)
AND l.borrowed < s.year;
```

An alternative is to use $\bowtie$ (i.e., **INNER JOIN**).

```
π(s.email)(
p(student, s) ⋈ π(s.email = l.borrower ∨ s.email = l.owner) ∧ (l.borrowed < s.year) p(loan, l)
)
```

Code: Alternative #2: INNER JOIN

```
SELECT s.email
FROM student s
INNER JOIN loan l ON (s.email = l.borrower OR s.email = l.owner)
AND l.borrowed < s.year;
```

We can also combine the $\bowtie$ with σ by moving some of the conditions around. This is fine as $\bowtie$ corresponds to **INNER JOIN**. Such technique may not be possible for **OUTER JOIN**. Finally, we can use **UNION**.

```
π(s1.email)(
σ(s1.email = l1.borrower ∧ l1.borrowed < s1.year)(p(student, s1) × p(loan, l1)))
∪
π(s2.email)(
σ(s2.email = l2.owner ∧ l2.borrowed < s2.year)(p(student, s2) × p(loan, l2)))
)
```

Code: Alternative #3: UNION

```
SELECT s1.email
FROM loan l1, student s1
WHERE s1.email = l1.borrower
AND l1.borrowed < s1.year
UNION
SELECT s2.email
FROM loan l2, student s2
WHERE s2.email = l2.borrower
AND l2.borrowed < s2.year
```

(c) Print the emails of the students who borrowed but did not lend a copy of a book on the day that they joined the university.

Comments: We use non-symmetric set difference. There are two convention for this operation. The first is $x - y$ and the second is $x \setminus y$. Both mean the same thing. Our convention is typically the first (i.e., $x - y$).

```
π(s1.email)(
σ(s1.email = l1.borrower ∧ l1.borrowed < s1.year)(p(student, s1) × p(loan, l1)))
-
π(s2.email)(
σ(s2.email = l2.owner ∧ l2.borrowed < s2.year)(p(student, s2) × p(loan, l2)))
)
```

Code: Query

```
SELECT s1.email
FROM loan l1, student s1
WHERE s1.email = l1.borrower
AND l1.borrowed < s1.year
EXCEPT
SELECT s2.email
FROM loan l2, student s2
WHERE s2.email = l2.borrower
AND l2.borrowed < s2.year
```

Code: Query from Relational Algebra

```
SELECT s3.email, s3.name
FROM student s3
EXCEPT
SELECT tmp.email, tmp.name
FROM (
  SELECT s1.email, s1.name, b1.ISBN13
  FROM student s1, book b1
```

Code: Query

```
SELECT d.department
FROM department d
WHERE d.faculty = 'School of Computing'
```

Code: Query

```
SELECT s1.email, s2.name, b2.ISBN13
FROM student s2, book b2, loan l2
WHERE b2.authors = 'Adam Smith'
AND s2.email = l2.borrower
AND b2.ISBN13 = l2.book
) AS tmp;
```